

Real-Time E-Commerce Order Tracking Platform

Team Members

Murali Manohara and **Soundhariya V**

1. Problem Statement

A fast-growing online retailer wants to provide customers with fast and reliable order-tracking experience. Customers should be able to see order status changes such as **Packed, Shipped, Out for Delivery, and Delivered** with minimal delay.

As order volumes increase, especially during major sales events, the existing system may face problems such as:

- Order updates arriving late.
- Some status updates are missing.
- High traffic causes services to become overloaded.
- Failures in one service affecting other services.
- Delays in sending notifications.

The main objective is to design a system that is:

- **Fast** – provides near real-time order updates.
 - **Reliable** – minimizes missed or lost updates.
 - **Scalable** – handles sudden increases in traffic.
 - **Secure** – protects customer communication.
 - **Highly available** – continues working even if individual components fail.
-

2. Assumptions

The following assumptions are made regarding our proposed solution:

1. Customers access the application through a web or mobile application.
2. All customer-facing communication uses **HTTPS/TLS**.

3. PostgreSQL is used to store order information and the current order status.
 4. Order status can change through stages such as:
Placed → Processing → Packed → Shipped → Out for Delivery → Delivered
 5. The system may experience sudden traffic increases during flash sales or festival sales.
 6. WebSocket is used to provide real-time updates to customers.
 7. Kafka is used to communicate order-status events between backend services.
 8. Multiple instances of important services can run to improve availability and scalability.
-

3. Proposed Solution

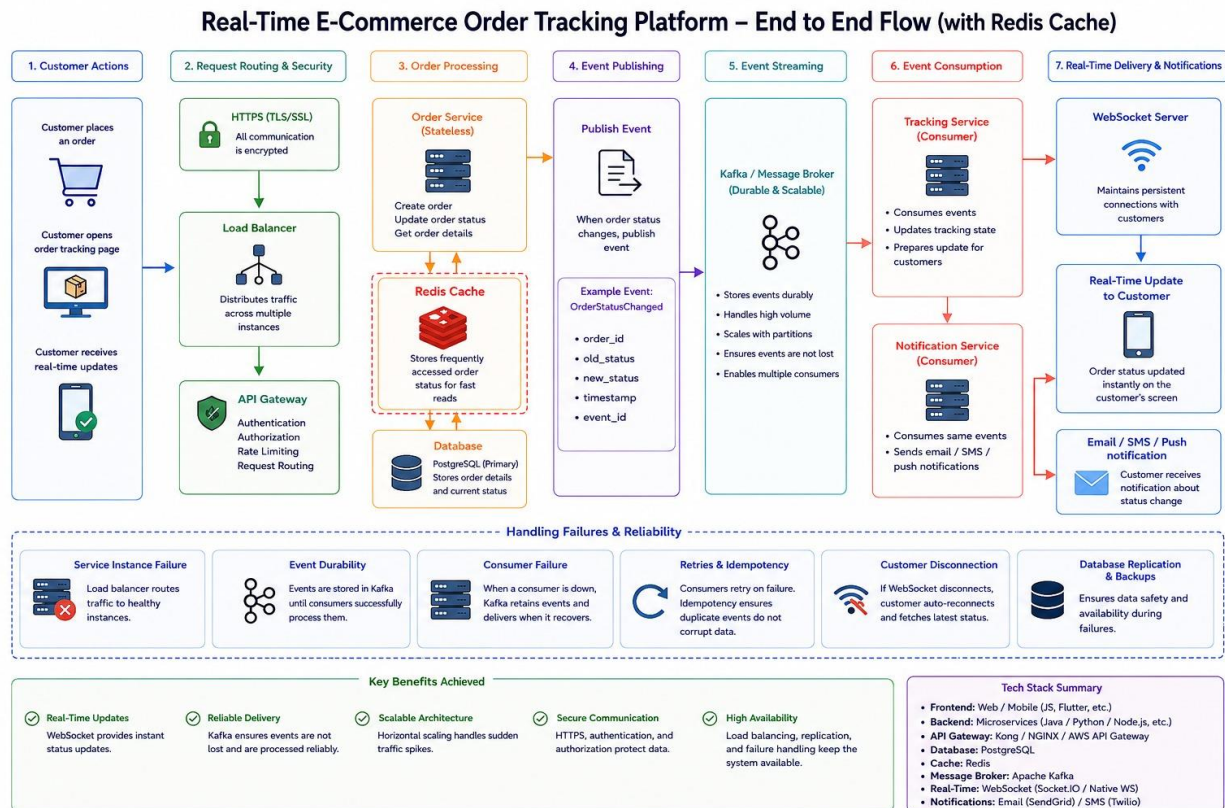
We propose **distributed and event-driven architecture**.

The main components are:

- **Load Balancer** – distributes incoming requests across multiple application instances.
- **API Gateway** – handles routing, authentication, authorization, and rate limiting.
- **Order Service** – manages order creation and status changes.
- **PostgreSQL** – stores the current order state.
- **Kafka** – carries order-status events between services.
- **Tracking Service** – processes order events for real-time tracking.
- **WebSocket** – pushes status updates directly to connected customers.
- **Notification Service** – sends email, SMS, or push notifications.

4. Architecture Diagram

End-to-End Architecture Flow



The flowchart represents the complete flow from the customer to the backend services and finally back to the customer.

Step-by-Step Flow

Step 1 – Customer Request

The customer opens the application or performs an order-related action. The request is sent using **HTTPS**, which encrypts communication.

Step 2 – Load Balancer

The Load Balancer distributes requests across multiple application instances. This prevents one server from receiving all the traffic and helps the system handle traffic spikes.

Step 3 – API Gateway

The API Gateway acts as the entry point to the backend. It performs functions such as request routing, authentication, authorization, and rate limiting.

Step 4 – Order Service

The Order Service handles order operations such as creating an order, updating its status, and retrieving order details.

Step 5 – Database

The current order status is stored in PostgreSQL.

For example:

Order ID: 1001

Status: Shipped

The database remains the source of the current order state.

Step 6 – Event Publishing

When an order status changes, the Order Service publishes an `OrderStatusChanged` event.

Example:

Order ID: 1001

Old Status: Packed

New Status: Shipped

Event ID: E10001

Step 7 – Kafka

Kafka receives the event and makes it available to the required services.

This separates the Order Service from the Tracking and Notification Services.

Step 8 – Tracking Service

The Tracking Service consumes the event and prepares the latest status for the customer.

Step 9 – WebSocket

WebSocket pushes the update to the customer's browser or mobile application without requiring the customer to refresh the page.

Step 10 – Notification Service

The Notification Service can consume the same event and send an email, SMS, or push notification.

5. Key Design Decisions

5.1 HTTPS

We use **HTTPS instead of HTTP** for customer communication.

HTTPS uses TLS encryption, which protects information while it travels between the customer and the server.

It is important because the application handles customer and order information.

5.2 Load Balancer

We use a Load Balancer because traffic can increase suddenly during major sales.

Instead of:

Customers → One Server

we use:

Load Balancer

/ | \

↓ ↓ ↓

Server 1 Server 2 Server 3

This improves scalability and availability.

5.3 PostgreSQL

PostgreSQL is used because order information is structured and requires reliable, persistent storage.

It stores the latest state of each order.

For example:

Order 1001 → Shipped

Order 1002 → Packed

Order 1003 → Delivered

5.4 Kafka

Kafka is a distributed event-streaming platform. Kafka is a middle system that receives events from one service, stores them safely, and allows other services to read and process those events. **Order Service publishes an event** such as Order Status Changed to a Kafka topic. **Kafka stores the event durably** and distributes events across **partitions** for high throughput. **Tracking and Notification Services consume the events** and process them independently.

If a consumer fails, **Kafka retains the event**, allowing the consumer to retry it when it recovers.

Kafka allows services to process events independently.

Why not directly call the services?

If the Order Service directly depends on the Tracking and Notification Services, a failure in one service could affect order processing.

Kafka reduces this dependency and allows events to be retained for later processing.

5.5 WebSocket

WebSocket is used because customers need real-time order updates.

With traditional polling, the browser repeatedly asks:

"Has my order changed?" With WebSocket, the server can push the update when it happens: Tracking Service ——— WebSocket ———▶ Customer ———▶
"Order Shipped"

This reduces unnecessary requests and provides faster updates.

5.6 Separate Tracking and Notification Services

Tracking and notification processing are separated.

The Tracking Service focuses on **real-time customer updates**, while the Notification Service handles **email/SMS/push notifications**.

Therefore, if email processing becomes slow, it does not have to block the customer's real-time tracking update.

6. Reliability and Scalability

The proposed system uses several mechanisms to improve reliability.

Service Failure

If one application instance fails, the Load Balancer can route requests to healthy instances.

Event Failure

Kafka retains events according to its configured retention, allowing consumers to process events after recovering from temporary failures.

Duplicate Events

A unique **event ID** can be used to identify already-processed events. This provides idempotent processing and prevents the same event from incorrectly updating the system multiple times.

Customer Disconnection

If a customer's WebSocket connection is lost, the client can reconnect and retrieve the latest order status from the backend.

Traffic Spikes

Multiple instances of services can be added during high traffic. Kafka consumers can also be scaled to process more events.

Therefore, the system can handle both **high request traffic and high event volume**.

7. Demonstration Explanation

For the demonstration, we use a simple example:

Order #1001 changes from "Packed" to "Shipped".

Step 1

The order status is initially:

Order #1001

Status = Packed

Step 2

The Order Service receives a request to change the status.

Step 3

The Order Service validates the request and updates PostgreSQL:

Packed → Shipped

Step 4

After the successful update, the Order Service publishes:

OrderStatusChanged

to Kafka.

Step 5

Kafka makes the event available to the Tracking Service and Notification Service.

Step 6

The Tracking Service receives the event and sends the update through WebSocket.

Step 7

The customer's screen immediately shows:

Order #1001

Status: Shipped

Step 8

At the same time, the Notification Service can send:

Your order #1001 has been shipped.

The important point is that **the order state is stored in PostgreSQL, while Kafka communicates the change to other services.**

8. Handling Multiple Orders

The system can process multiple orders at the same time.

For example:

Order 101 → Shipped

Order 102 → Packed

Order 103 → Delivered

Each status change produces a separate event:

Event 1 → Order 101 → Shipped

Event 2 → Order 102 → Packed

Event 3 → Order 103 → Delivered

Kafka allows these events to be processed independently.

If multiple users try to modify the **same order simultaneously**, database concurrency control such as optimistic locking can prevent an outdated update from overwriting a newer update.

9. Why This Architecture Solves the Problem

Requirement	Our Solution
Fast order updates	WebSocket
Reliable event processing	Kafka
Secure communication	HTTPS/TLS
Sudden traffic increase	Load Balancer + horizontal scaling
Service failures	Multiple instances + recovery
Notification delays	Separate Notification Service
Duplicate events	Event ID + idempotency
Persistent order state	PostgreSQL
Loose coupling	Kafka-based event communication
High availability	Load balancing + redundancy

10. Conclusion

Our proposed solution uses distributed, **event-driven architecture** to provide reliable and responsive order-tracking experience.

The customer communicates securely through **HTTPS**. The **Load Balancer** distributes traffic, while the **API Gateway** routes and controls requests. The **Order Service** manages order operations and stores the current order state in **PostgreSQL**.

Whenever an order status changes, the Order Service publishes an event to **Kafka**. The **Tracking Service** consumes the event and uses **WebSocket** to provide a real-time update to the customer. The **Notification Service** independently processes the same event for email, SMS, or push notifications.

The use of load balancing, horizontal scaling, Kafka, retry, idempotency, and service redundancy helps the system remain reliable during failures and sudden increases in order activity.

Final Design Principle

PostgreSQL stores the current order state, Kafka communicates order-status changes, the Tracking Service processes real-time updates, WebSocket delivers them to customers, and the Notification Service handles notifications independently.

This design provides the required **speed, reliability, security, scalability, and availability** for a modern real-time e-commerce order-tracking platform.